# ROME Demo Assignment

## Hierarchical LLM-assisted Verilog generation with decomposition and verification

Course: LLM4ChipDesign | Instructors: Ramesh Karri, Weihua Xiao

## Overview

Use **ROME_demo.ipynb** to generate synthesizable Verilog with a **hierarchical decomposition** workflow. You will run the provided demo, extend it to a new hierarchical design, and verify correctness with **iVerilog** using self-checking testbenches.

## Tools and setup

- **Environment:** Google Colab (recommended) or local Jupyter.
- **Dependencies:** *openai*, *anthropic* (optional), *google-genai* (optional); *iVerilog* for compilation/simulation, *Yosys* for PPA optimization.
- **Models and keys:** choose ChatGPT / Claude / Gemini in the notebook and set the corresponding API key via environment variables or Colab Secrets. **Never** include keys in your submission.

## Provided demo example

**ROME_demo.ipynb** includes a single end-to-end example: a **Mux hierarchy** generated as submodules, then verified with testbenches.

- **mux2to1** (2-to-1 multiplexer) *through*
- mux64to1 (final 64-to-1 multiplexer assembled form intermediate sizes)

## What to submit

Submit a single ZIP (or a GitHub repo folder, if instructed) containing:

- **One runnable notebook: rome_submission.ipynb** (your completed version of ROME_demo.ipynb).

- **Generated RTL folders:** include the generated *.v* files, testbenches (**tb.v*), and iteration logs (*log_iter_*.txt*) for both the provided demo and your extension design.
- **A short report PDF: ROME_Report.pdf** (1–2 pages).

## Part I - Run the provided mux hierarchy demo (required)

Run **ROME_demo.ipynb** as-is and generate **mux2to1**, **mux4to1**, **mux8to1, mux16to1, mux32to1, and mux64to1**. Obtain passing testbenches for each module using the notebook's feedback loop. Run PPA optimizer and report best and worst transistor counts.

- **Model selection:** choose a model wrapper (ChatGPT / Claude / Gemini) and set the corresponding API key.
- **Reproducibility:** run the notebook top-to-bottom (except for inserting an API key in one obvious place).
- **Verification evidence:** show the output where each module reaches *Testbench ran successfully* (and the testbench prints *passed!*) as well as PPA optimization results.
- **Artifacts:** ensure each module folder (e.g., *mux2to1/*) contains the final *.v* plus logs generated by the loop.

## Part II - Extend to a new hierarchical RTL design (required)

Create a new hierarchical design by defining a **submodules** list (at least **3** modules, including a top module) and generating each module with the hierarchical loop. You must provide a **self-checking testbench** for each module.

Choose **one** extension option below (or propose an equivalent design of similar complexity):

- **Option A: Ripple-carry adder hierarchy** (recommended). Example decomposition: half_adder -> full_adder -> adder4 -> adder8.
- **Option B: Comparator hierarchy**. Example decomposition: cmp1 -> cmp4 -> cmp8 (or signed/unsigned variants).
- **Option C: Encoder/decoder hierarchy**. Example decomposition: pri_enc4 -> pri_enc8 -> top_wrapper (with valid flag).
- **Instantiation requirement:** your top module must instantiate at least **two** previously generated submodules (i.e., not a monolithic re-write).
- **Testbench requirement:** each testbench must print a clear final line containing the word *passed!* when successful (the notebook parses this).
- **Evidence:** show the final passing *iVerilog* output for each module & *Yosys* PPA optimization results.

## Part III - Document your debugging loop and results (required)

For at least **one** module from Part II, document an iterative improvement process with at least **three** iterations (compile error, warning, or failing test). Use the iteration logs generated by the notebook.

- **Iteration log:** for each iteration, note (a) what failed, (b) what you changed (prompt/code), (c) the outcome.
- **Include snippets:** include the key *iVerilog* error/output lines that motivated your fix.
- **End with a final passing run.**

## Report requirements (ROME_Report.pdf)

- State the model(s) used and any key generation settings (if changed).
- Describe your hierarchical decomposition (module list, interfaces, and which modules instantiate which).
- Include your final prompts for the top module and one representative submodule.
- Provide a concise iteration table for Part III: iteration number, issue observed, fix applied, outcome.
- Confirm all testbenches pass and list the final *iVerilog* commands used.

## Grading rubric (suggested)

| Criterion | Points |
|---|---|
| Correctness: provided mux demo passes all testbenches | 25 |
| Correctness: your extension design passes all testbenches | 35 |
| Hierarchy quality: meaningful decomposition + instantiation of submodules | 20 |
| Debugging documentation + clarity & reproducibility (notebook + report, include PPA) | 20 |

## Academic integrity and responsible use

Using an LLM is required for this assignment, but you must submit your own work. Do not share final prompts/code between teams. Always verify generated RTL with simulation, and treat the LLM as a tool that can be wrong. If you use external references, cite them briefly in your report.